Q1_FFNN_CIFAR10.py

```python
# Q1: Feed Forward Neural Network for classification using CIFAR-10
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense
from tensorflow.keras.utils import to_categorical

tf.keras.utils.set_random_seed(42)

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
    Input(shape=(32, 32, 3)),
    Flatten(),
    Dense(512, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])

model.fit(x_train, y_train, epochs=15, batch_size=128,
          validation_split=0.1, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")
print("Experiment completed successfully!")
```

Q2_FFNN_MNIST.py

```python
# Q2: FFNN for handwritten digit classification using MNIST
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(x_train, y_train, epochs=8, batch_size=128,
                    validation_split=0.1, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q2_accuracy.png")
plt.show()
print("Experiment completed successfully!")
```

Q3_FFNN_Activation_Comparison.py

```python
# Q3: Compare ReLU, Sigmoid and Tanh activation functions in an FFNN
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Use the same subset and training settings for each activation.
x_train = x_train[:20000]
y_train = y_train[:20000]

activations = ["relu", "sigmoid", "tanh"]
results = []

for activation in activations:
    print(f"\nTraining with {activation} activation")
    tf.keras.utils.set_random_seed(42)

    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation=activation),
        Dense(10, activation="softmax")
    ])

    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    history = model.fit(
        x_train, y_train, epochs=5, batch_size=128,
        validation_split=0.1, verbose=1
    )

    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
    results.append((activation, test_accuracy, test_loss))
    print(f"Test accuracy: {test_accuracy * 100:.2f}%")
    print(f"Test loss: {test_loss:.4f}")

names = [item[0] for item in results]
accuracies = [item[1] * 100 for item in results]

print("\nActivation\tTest accuracy\tTest loss")
for name, accuracy, loss in results:
    print(f"{name}\t\t{accuracy:.4f}\t\t{loss:.4f}")

plt.figure(figsize=(8, 5))
plt.bar(names, accuracies)
plt.title("Test Accuracy for Different Activation Functions")
plt.xlabel("Activation function")
plt.ylabel("Test accuracy (%)")
plt.ylim(0, 100)
plt.grid(axis="y")
```

```
plt.tight_layout()
plt.savefig("Q3_activation_comparison.png")
plt.show()
print("Experiment completed successfully!")
```

Q4_FFNN_Xavier_Kaiming.py

```python
# Q4: Compare Xavier and Kaiming initialization in an FFNN
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Smaller subset keeps execution time manageable.
x_train = x_train[:20000]
y_train = y_train[:20000]

initializers = {
    "Xavier": "glorot_uniform",
    "Kaiming": "he_normal"
}
histories = {}
results = {}

for name, initializer in initializers.items():
    print(f"\nTraining with {name} initialization")
    tf.keras.utils.set_random_seed(42)

    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation="relu", kernel_initializer=initializer),
        Dense(10, activation="softmax")
    ])

    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    history = model.fit(
        x_train, y_train, epochs=8, batch_size=128,
        validation_split=0.1, verbose=1
    )
    histories[name] = history.history
    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
    results[name] = (test_loss, test_accuracy)
    print(f"Test accuracy: {test_accuracy * 100:.2f}%")
    print(f"Test loss: {test_loss:.4f}")

plt.figure(figsize=(8, 5))
for name in initializers:
    plt.plot(histories[name]["loss"], label=f"{name} training")
plt.title("Training Loss: Xavier vs Kaiming")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q4_training_loss.png")
```

```
plt.show()

plt.figure(figsize=(8, 5))
for name in initializers:
    plt.plot(histories[name]["val_loss"], label=f"{name} validation")
plt.title("Validation Loss: Xavier vs Kaiming")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q4_validation_loss.png")
plt.show()

print("\nMethod\t\tTest accuracy\tTest loss")
for name, (loss, accuracy) in results.items():
    print(f"{name}\t\t{accuracy * 100:.2f}%\t\t{loss:.4f}")

best = min(histories, key=lambda key: min(histories[key]["val_loss"]))
print("Lowest validation loss during training:", best)
print("Compare the curves; do not assume one initializer always wins.")
print("Experiment completed successfully!")
```

Q5_CNN_MNIST_LeNet.py

```python
# Q5: LeNet-like CNN for MNIST handwritten digit classification
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(6, (5, 5), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(16, (5, 5), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(120, activation="relu"),
    Dense(84, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy"])

model.fit(x_train, y_train_cat, epochs=10, batch_size=64,
          validation_split=0.2, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test_cat, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

predictions = model.predict(x_test, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(15, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Predicted: {predicted_labels[i]}\nActual: {y_test[i]}")
    plt.axis("off")
plt.tight_layout()
plt.savefig("Q5_sample_predictions.png")
plt.show()
print("Experiment completed successfully!")
```

Q7_FFNN_Xavier_Kaiming.py

```python
# Q7: Compare Xavier and Kaiming initialization in a neural network
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train[:20000]
y_train = y_train[:20000]

initializers = {"Xavier": "glorot_uniform", "Kaiming": "he_normal"}
histories = {}
results = {}

for name, initializer in initializers.items():
    print(f"\nTraining using {name} initialization")
    tf.keras.utils.set_random_seed(42)
    model = Sequential([
        Input(shape=(28, 28)),
        Flatten(),
        Dense(128, activation="relu", kernel_initializer=initializer),
        Dense(64, activation="relu", kernel_initializer=initializer),
        Dense(10, activation="softmax")
    ])
    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    history = model.fit(x_train, y_train, epochs=8, batch_size=128,
                        validation_split=0.1, verbose=1)
    histories[name] = history.history
    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
    results[name] = (test_loss, test_accuracy)
    print(f"Test accuracy: {test_accuracy * 100:.2f}%")

for metric, title, filename in [
    ("accuracy", "Training Accuracy Comparison", "Q7_training_accuracy.png"),
    ("loss", "Training Loss Comparison", "Q7_training_loss.png")
]:
    plt.figure(figsize=(8, 5))
    for name in initializers:
        plt.plot(histories[name][metric], label=name)
    plt.title(title)
    plt.xlabel("Epoch")
    plt.ylabel(metric.capitalize())
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(filename)
    plt.show()

print("\nMethod\tTest accuracy\tTest loss")
for name, (loss, accuracy) in results.items():
    print(f"{name}\t{accuracy * 100:.2f}%\t\t{loss:.4f}")
print("Experiment completed successfully!")
```

Q8_CNN_Architecture_MNIST.py

```python
# Q8: CNN architecture with convolution, pooling, flatten and dense layers
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense

tf.keras.utils.set_random_seed(42)
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

model.summary()
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

history = model.fit(x_train, y_train, epochs=10, batch_size=64,
                    validation_split=0.2, verbose=1)

test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training accuracy")
plt.plot(history.history["val_accuracy"], label="Validation accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("Q8_accuracy.png")
plt.show()
print("Experiment completed successfully!")
```

Q9_CNN_Prediction_Visualization.py

```python
# Q9: CNN for MNIST digit recognition; show correct and incorrect predictions
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense

tf.keras.utils.set_random_seed(42)

# 1. Load and preprocess MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

# 2. Build CNN
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(10, activation="softmax")
])

# 3. Compile and train
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(x_train, y_train, epochs=5, batch_size=64,
          validation_split=0.2, verbose=1)

# 4. Evaluate
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

# 5. Predict test images
probabilities = model.predict(x_test, verbose=0)
predicted_labels = np.argmax(probabilities, axis=1)

correct_indices = np.where(predicted_labels == y_test)[0]
incorrect_indices = np.where(predicted_labels != y_test)[0]

print("Correctly classified images:", len(correct_indices))
print("Incorrectly classified images:", len(incorrect_indices))

# 6. Display correctly classified examples
if len(correct_indices) > 0:
    selected = correct_indices[:10]
    plt.figure(figsize=(12, 5))
    for plot_number, index in enumerate(selected, start=1):
        plt.subplot(2, 5, plot_number)
        plt.imshow(x_test[index].squeeze(), cmap="gray")
        plt.title(f"Pred: {predicted_labels[index]}\nActual: {y_test[index]}")
```

```
        plt.axis("off")
    plt.suptitle("Correctly Classified Images")
    plt.tight_layout()
    plt.savefig("Q9_correct_predictions.png")
    plt.show()

# 7. Display incorrectly classified examples
if len(incorrect_indices) > 0:
    selected = incorrect_indices[:10]
    plt.figure(figsize=(12, 5))
    for plot_number, index in enumerate(selected, start=1):
        plt.subplot(2, 5, plot_number)
        plt.imshow(x_test[index].squeeze(), cmap="gray")
        plt.title(f"Pred: {predicted_labels[index]}\nActual: {y_test[index]}")
        plt.axis("off")
    plt.suptitle("Incorrectly Classified Images")
    plt.tight_layout()
    plt.savefig("Q9_incorrect_predictions.png")
    plt.show()
else:
    print("No incorrectly classified images were found.")

print("Experiment completed successfully!")
```

Q14 RNN LSTM IMDb

```
# Question 14: RNN vs LSTM IMDb training and validation accuracy
# Requires: tensorflow numpy matplotlib scikit-learn

import time
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Embedding, SimpleRNN, LSTM, GRU, Dropout, Dense
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import confusion_matrix, classification_report

tf.keras.utils.set_random_seed(42)
VOCAB_SIZE = 10000
MAX_LEN = 200  # Change to 500 to match the full Experiment 7 setting
EPOCHS = 5    # Increase to 12 for the full lab setting
BATCH_SIZE = 64

# Use the SAME train/test split for every model in comparisons.
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)
x_train = pad_sequences(x_train, maxlen=MAX_LEN)
x_test = pad_sequences(x_test, maxlen=MAX_LEN)


def build_model(layer_type):
    layers = {'SimpleRNN': SimpleRNN, 'LSTM': LSTM, 'GRU': GRU}
    model = Sequential([
        Input(shape=(MAX_LEN,)),
        Embedding(input_dim=VOCAB_SIZE, output_dim=32, mask_zero=True),
        layers[layer_type](64),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001, clipnorm=1.0),
                  loss='binary_crossentropy', metrics=['accuracy'])
    return model


def train_evaluate(name):
    print('\nTraining', name)
    model = build_model(name)
    model.summary()
    start = time.perf_counter()
    history = model.fit(x_train, y_train, epochs=EPOCHS,
                        batch_size=BATCH_SIZE, validation_split=0.2,
                        callbacks=[EarlyStopping(monitor='val_loss', patience=2,
                                                 restore_best_weights=True)], verbose=1)
    seconds = time.perf_counter() - start
    loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
    print(f'{name}: test accuracy={accuracy:.4f}, test loss={loss:.4f}, training seconds={second
s:.1f}')
    return model, history, loss, accuracy, seconds


def plot_history(histories, metric):
    plt.figure(figsize=(9, 5))
    for name, history in histories.items():
        plt.plot(history.history[metric], label=f'{name} train')
        plt.plot(history.history['val_' + metric], '--', label=f'{name} validation')
    plt.title('Training and Validation ' + metric.title())
    plt.xlabel('Epoch')
    plt.ylabel(metric.title())
```

```
    plt.legend()
    plt.tight_layout()
    plt.show()


def show_classification(model, name):
    pred = (model.predict(x_test, verbose=0).ravel() >= 0.5).astype(int)
    cm = confusion_matrix(y_test, pred)
    print('\nConfusion matrix for', name, '\n', cm)
    print(classification_report(y_test, pred, target_names=['Negative', 'Positive']))
    plt.figure(figsize=(5, 4))
    plt.imshow(cm, cmap='Blues')
    plt.title(name + ' Confusion Matrix')
    plt.xticks([0, 1], ['Negative', 'Positive'])
    plt.yticks([0, 1], ['Negative', 'Positive'])
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    for i in range(2):
        for j in range(2):
            plt.text(j, i, str(cm[i, j]), ha='center', va='center')
    plt.colorbar()
    plt.tight_layout()
    plt.show()


if __name__ == "__main__":
    results = {}
    for name in ['SimpleRNN', 'LSTM']:
        model, history, loss, accuracy, seconds = train_evaluate(name)
        results[name] = (history, loss, accuracy, seconds)
        print(f'{name} final training accuracy: {history.history["accuracy"][-1]:.4f}')
        print(f'{name} final validation accuracy: {history.history["val_accuracy"][-1]:.4f}')
    plot_history({name: v[0] for name, v in results.items()}, 'accuracy')
    plot_history({name: v[0] for name, v in results.items()}, 'loss')
    print('Explanation: LSTM has input, forget, and output gates plus a cell state,')
    print('which can help retain useful information over longer sequences.')
    print('Actual performance depends on training, data, and hyperparameters.')
```

Q15 RNN vs GRU

```
# Question 15: Compare Simple RNN and GRU accuracy loss and training time
# Requires: tensorflow numpy matplotlib scikit-learn

import time
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Embedding, SimpleRNN, LSTM, GRU, Dropout, Dense
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import confusion_matrix, classification_report

tf.keras.utils.set_random_seed(42)
VOCAB_SIZE = 10000
MAX_LEN = 200  # Change to 500 to match the full Experiment 7 setting
EPOCHS = 5    # Increase to 12 for the full lab setting
BATCH_SIZE = 64

# Use the SAME train/test split for every model in comparisons.
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)
x_train = pad_sequences(x_train, maxlen=MAX_LEN)
x_test = pad_sequences(x_test, maxlen=MAX_LEN)


def build_model(layer_type):
    layers = {'SimpleRNN': SimpleRNN, 'LSTM': LSTM, 'GRU': GRU}
    model = Sequential([
        Input(shape=(MAX_LEN,)),
        Embedding(input_dim=VOCAB_SIZE, output_dim=32, mask_zero=True),
        layers[layer_type](64),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001, clipnorm=1.0),
                  loss='binary_crossentropy', metrics=['accuracy'])
    return model


def train_evaluate(name):
    print('\nTraining', name)
    model = build_model(name)
    model.summary()
    start = time.perf_counter()
    history = model.fit(x_train, y_train, epochs=EPOCHS,
                        batch_size=BATCH_SIZE, validation_split=0.2,
                        callbacks=[EarlyStopping(monitor='val_loss', patience=2,
                                                 restore_best_weights=True)], verbose=1)
    seconds = time.perf_counter() - start
    loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
    print(f'{name}: test accuracy={accuracy:.4f}, test loss={loss:.4f}, training seconds={second
s:.1f}')
    return model, history, loss, accuracy, seconds


def plot_history(histories, metric):
    plt.figure(figsize=(9, 5))
    for name, history in histories.items():
        plt.plot(history.history[metric], label=f'{name} train')
        plt.plot(history.history['val_' + metric], '--', label=f'{name} validation')
    plt.title('Training and Validation ' + metric.title())
    plt.xlabel('Epoch')
    plt.ylabel(metric.title())
```

```
    plt.legend()
    plt.tight_layout()
    plt.show()


def show_classification(model, name):
    pred = (model.predict(x_test, verbose=0).ravel() >= 0.5).astype(int)
    cm = confusion_matrix(y_test, pred)
    print('\nConfusion matrix for', name, '\n', cm)
    print(classification_report(y_test, pred, target_names=['Negative', 'Positive']))
    plt.figure(figsize=(5, 4))
    plt.imshow(cm, cmap='Blues')
    plt.title(name + ' Confusion Matrix')
    plt.xticks([0, 1], ['Negative', 'Positive'])
    plt.yticks([0, 1], ['Negative', 'Positive'])
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    for i in range(2):
        for j in range(2):
            plt.text(j, i, str(cm[i, j]), ha='center', va='center')
    plt.colorbar()
    plt.tight_layout()
    plt.show()


if __name__ == "__main__":
    results = {}
    for name in ['SimpleRNN', 'GRU']:
        model, history, loss, accuracy, seconds = train_evaluate(name)
        results[name] = (history, loss, accuracy, seconds)
    print('\nMODEL COMPARISON')
    for name, (_, loss, accuracy, seconds) in results.items():
        print(f'{name}: accuracy={accuracy:.4f}, loss={loss:.4f}, training time={seconds:.1f}s')
    plot_history({name: v[0] for name, v in results.items()}, 'accuracy')
    plot_history({name: v[0] for name, v in results.items()}, 'loss')
```